

키와 기본 명령어

Redis UI 접속

 docker.desktop

PERSONAL

Ctrl+K

     G — □ ×

Containers [Give feedback](#)

Container CPU usage ⓘ
0.51% / 1600% (16 CPUs available)

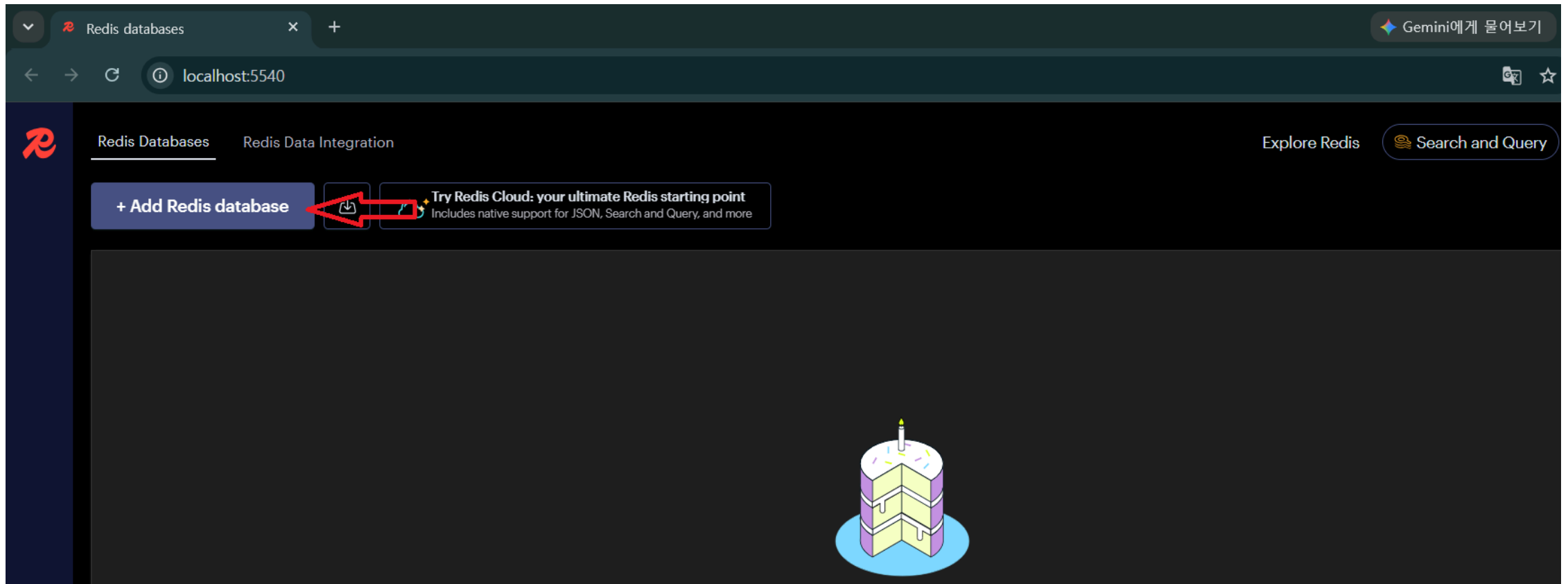
Container memory usage ⓘ
380.4MB / 14.87GB

 ☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Actions
☐ 	 redis	-	-	-	0.25%	227.7MB / 30.44%	  
☐	 insight	57cd4f68697c	redis/redis	5540:5540 	0%	103.1MB / 15.22%	  
☐	 stack	93a5ec88dc28	redis/redis	6379:6379  Show all ports (2)	0.25%	124.6MB / 15.22%	  

2

Add Redis database



- Host : redis
- Port : 6379
- Database Alias : tutorial

Redis Databases Redis Data Integration Explore Redis Search and Query JSON

+ Add Redis database

Try Redis Cloud: your ultimate Redis starting point
Includes native support for JSON, Search and Query, and more

Discover and Add Redis Databases

**Add Database Manually**
Use Host and Port to connect to your Redis Database

**Autodiscover Databases**
Use discovery tools to automatically discover and add your Redis Databases

You can manually add your Redis databases. Enter Host and Port of your Redis database to add it to Redis Insight. [Learn more here.](#)

Host* 

Port*
Should not exceed 65535.

Database Alias*

Username Password Timeout (s)

Test Connection Cancel Add Redis Database

Connect database

The screenshot shows the Redis Databases web interface. The browser address bar displays 'localhost:5540'. The interface includes a sidebar with navigation icons and a main content area with tabs for 'Redis Databases' and 'Redis Data Integration'. A '+ Add Redis database' button is visible. Below it, a table lists the connected databases. A red arrow points to the 'tutorial' database entry in the 'Database Alias' column.

☐	Database Alias	Host:Port	Connection Type	Modules	Last connection ↑
☐	<u>tutorial</u>	redis:6379	Standalone		less than a minute ago

학습 목표

- 일관된 규칙으로 Redis 키를 설계할 수 있습니다.
- 키의 존재 여부, 자료형, 삭제 여부를 확인할 수 있습니다.
- `KEYS` 와 `SCAN` 의 차이를 설명할 수 있습니다.

1. 키 이름 설계

Redis에는 테이블이 없으므로 키 이름이 데이터의 의미를 전달해야 합니다.
이 강의에서는 다음 규칙을 사용합니다.

서비스:대상:식별자[:속성]

shop:product:1001
shop:product:1001:views
community:user:7:session

좋은 키의 조건

- 의미가 분명하고 충돌하지 않습니다.
- 너무 짧거나 지나치게 길지 않습니다.
- 같은 종류의 키는 같은 패턴을 사용합니다.
- 개인정보를 키 이름에 직접 넣지 않습니다.

2. 기본 저장과 조회

SET shop:product:1001 "무선 키보드"

The screenshot shows a database management interface with a dark theme. On the left, a sidebar contains icons for databases, keys, a document (highlighted with a red box), charts, and a network icon. The main area displays the command `SET shop:product:1001 "무선 키보드"` in a text input field, which is also highlighted with a red box. To the right of the input field is a green play button, also highlighted with a red box. The top of the interface shows the database name 'tutorial', a dropdown for 'db0', and various status indicators like '0.28 %', '0', '2 MB', '0', and '4'. Below the input field, the text 'No results to display yet' is visible.

The screenshot shows the same database management interface after the command has been executed. The command `SET shop:product:1001 "무선 키보드"` is now in the command history at the bottom. The main area shows a single line of output: `"OK"`. The top status indicators now show '0.25 %', '0', '2 MB', '1', and '4'. A 'Clear Results' button is visible in the top right of the main area. The command history at the bottom includes the command, the timestamp 'Jun 19, 15:35:35', the execution time '3.483 msec', and icons for copy, delete, and other actions.

GET shop:product:1001

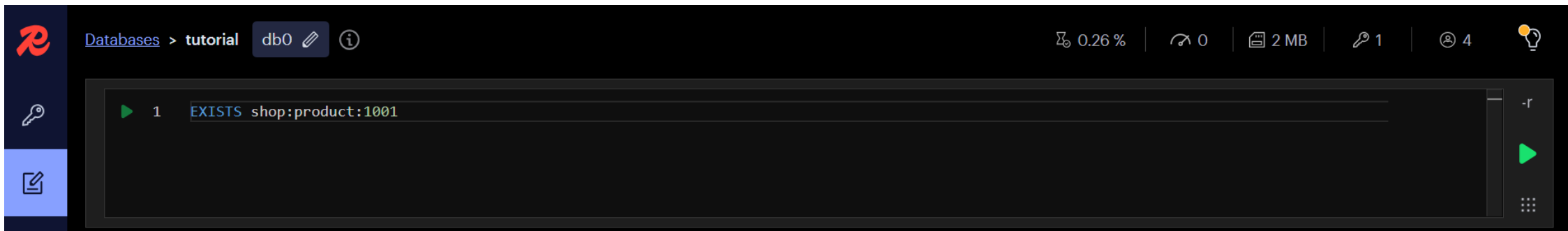
The screenshot shows a database client interface with a dark theme. The top navigation bar includes the text "Databases > tutorial db0" and several status icons: a gauge showing "0.27 %", a refresh icon with "0", a document icon with "2 MB", a key icon with "1", and a user icon with "4".

The main editor area contains a single query: `1 GET shop:product:1001`. To the right of the editor is a vertical toolbar with a play button and a grid icon.

Below the editor, the results are displayed in a table. The table has one row with the value "무선 키보드". Above the table is a "Clear Results" button. Below the table, a status bar shows the query "GET shop:product:1001", the timestamp "Jun 19, 15:38:05", the execution time "2.751 msec", and several icons for formatting and execution.

- EXISTS key : 키가 있으면 1, 없으면 0

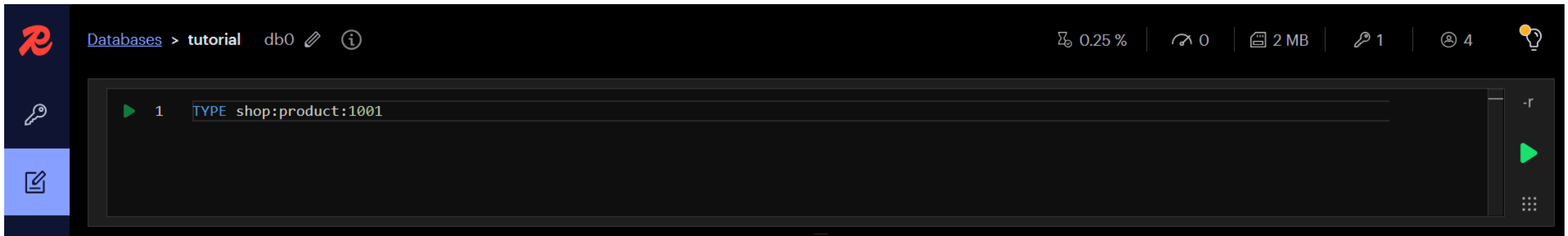
EXISTS shop:product:1001



Clear Results			
EXISTS shop:product:1001	Jun 19, 15:41:03	1.465 msec	   
(integer) 1			
GET shop:product:1001	Jun 19, 15:38:05	2.751 msec	  
SET shop:product:1001 "무선 키보드"	Jun 19, 15:35:35	3.483 msec	  

- TYPE key : 키에 저장된 자료형 확인

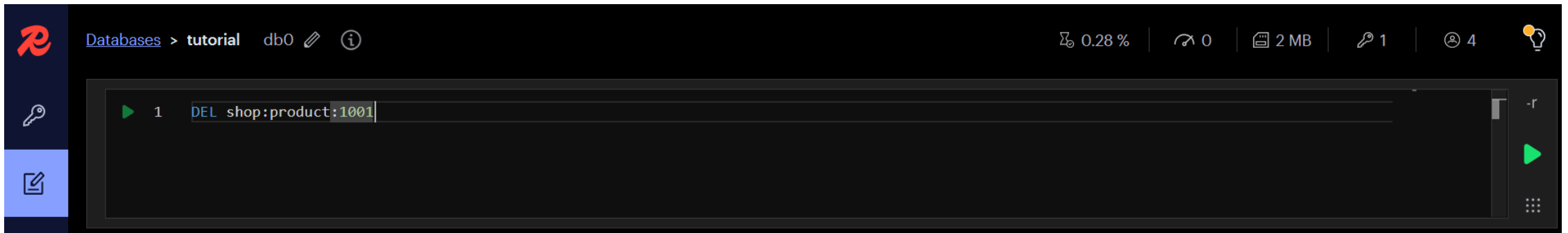
TYPE shop:product:1001



TYPE shop:product:1001				Jun 19, 15:42:28	0.943 msec	Clear Results		
"string"								
EXISTS shop:product:1001				Jun 19, 15:41:03	1.465 msec			
GET shop:product:1001				Jun 19, 15:38:05	2.751 msec			

- `DEL key [key ...]` : 하나 이상의 키를 즉시 삭제

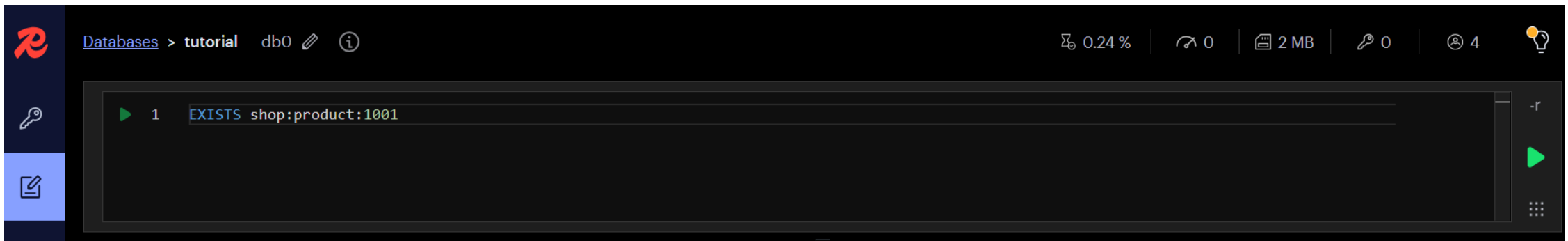
```
DEL shop:product:1001
```



Clear Results			
DEL shop:product:1001	Jun 19, 15:43:37	2.112 msec	   
(integer) 1			
TYPE shop:product:1001	Jun 19, 15:42:28	0.943 msec	  
EXISTS shop:product:1001	Jun 19, 15:41:03	1.465 msec	  

- EXISTS key : 키가 있으면 1, 없으면 0

EXISTS shop:product:1001



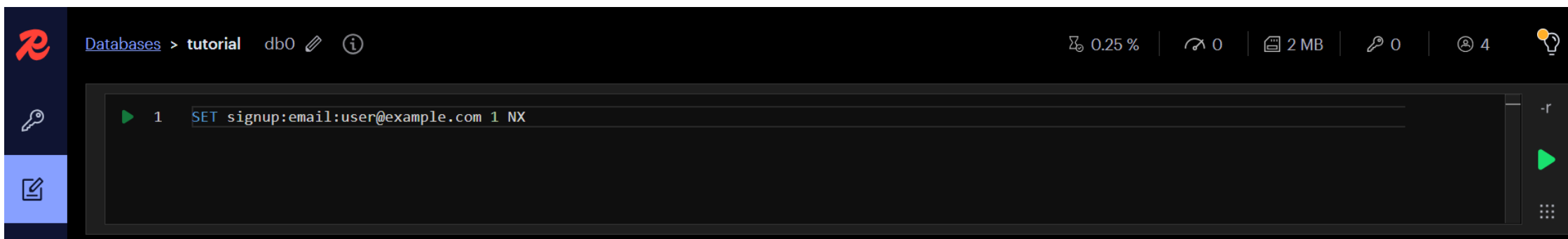
				Clear Results			
EXISTS shop:product:1001	Jun 19, 15:44:31	1.822 msec					
(integer) 0							
DEL shop:product:1001	Jun 19, 15:43:37	2.112 msec					
TYPE shop:product:1001	Jun 19, 15:42:28	0.943 msec					

3. SET 옵션

조건을 만족하지 못한 SET 은 (nil) 을 반환합니다. NX 는 간단한 중복 방지에 유용하지만, 복잡한 분산 잠금은 만료와 소유권 확인까지 별도로 설계해야 합니다.

- **NX** : 키가 없을 때만 저장

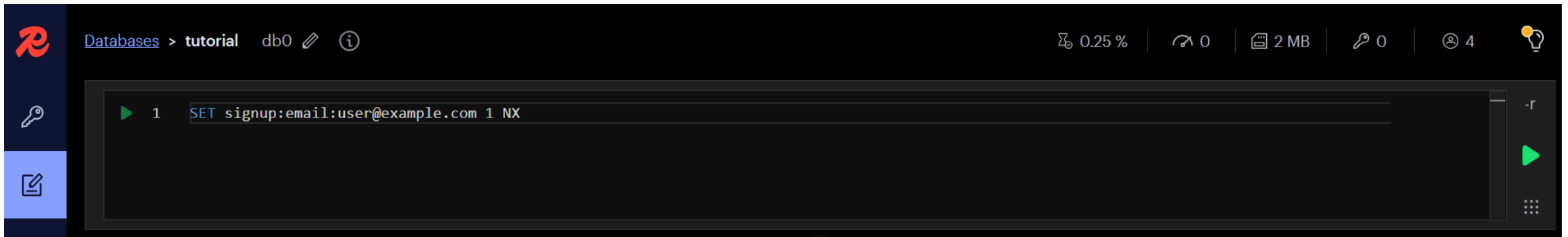
```
SET signup:email:user@example.com 1 NX
```



Clear Results			
SET signup:email:user@example.com 1 NX	Jun 19, 15:47:29	1.655 msec	   
"OK"			
EXISTS shop:product:1001	Jun 19, 15:44:31	1.822 msec	  
DEL shop:product:1001	Jun 19, 15:43:37	2.112 msec	  

- **NX**: 키가 없을 때만 저장

```
SET signup:email:user@example.com 1 NX
```

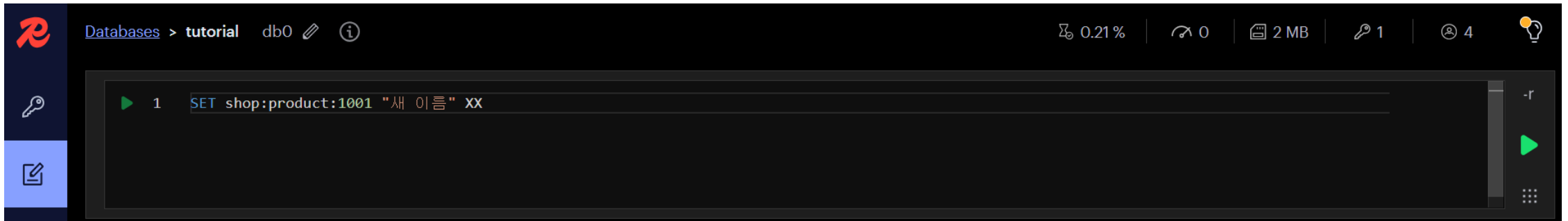


이미 저장된 키(signup:email:user@example.com)가 있기 때문에 저장 실패

Clear Results			
SET signup:email:user@example.com 1 NX	Jun 19, 16:01:00	1.872 msec	   
(nil)			
SET signup:email:user@example.com 1 NX	Jun 19, 15:47:29	1.655 msec	  
EXISTS shop:product:1001	Jun 19, 15:44:31	1.822 msec	  

- `xx`: 키가 있을 때만 저장

```
SET shop:product:1001 "새 이름" xx
```



저장된 키(shop:product:1001)가 없기 때문에 저장 실패

Clear Results			
SET shop:product:1001 "새 이름" xx	Jun 19, 16:03:49	0.8 msec	   
(nil)			
SET signup:email:user@example.com 1 NX	Jun 19, 16:01:00	1.872 msec	  
SET signup:email:user@example.com 1 NX	Jun 19, 15:47:29	1.655 msec	  

- EX 초 : 저장과 동시에 초 단위 만료 시간 설정
- PX 밀리초 : 저장과 동시에 밀리초 단위 만료 시간 설정

```
SET auth:code:1001 482913 EX 180
```

The screenshot shows a database management interface with a dark theme. The top bar indicates the current database is 'tutorial' (db0). The main area shows a single command being executed: `SET auth:code:1001 482913 EX 180`. Below the command, the results are displayed in a table format. The first result is `"OK"`, indicating the command was successful. The second result shows a command `SET shop:product:1001 "새 이름" XX` with a timestamp of `Jun 19, 16:03:49` and a duration of `0.8 msec`. The third result shows a command `SET signup:email:user@example.com 1 NX` with a timestamp of `Jun 19, 16:01:00` and a duration of `1.872 msec`. The interface includes various icons for navigation and actions, such as a play button, a trash icon, and a refresh icon.

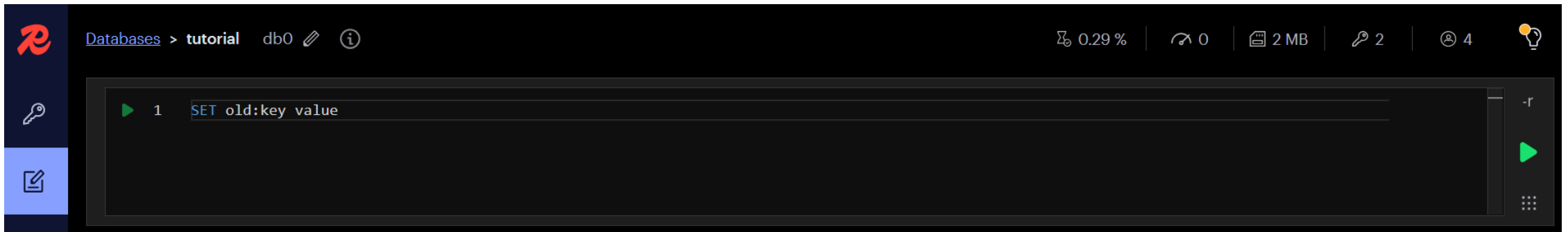
Command	Timestamp	Duration	Actions
SET auth:code:1001 482913 EX 180	Jun 19, 16:08:57	2.064 msec	[Copy] [Trash] [Play] [Up Arrow]
"OK"			
SET shop:product:1001 "새 이름" XX	Jun 19, 16:03:49	0.8 msec	[Copy] [Trash] [Play] [Down Arrow]
SET signup:email:user@example.com 1 NX	Jun 19, 16:01:00	1.872 msec	[Copy] [Trash] [Play] [Down Arrow]

4. 키 이름 변경

`RENAME` 은 대상 키가 이미 있으면 덮어씁니다. 덮어쓰기를 막으려면 `RENAMENX` 를 사용합니다.

- 키 생성

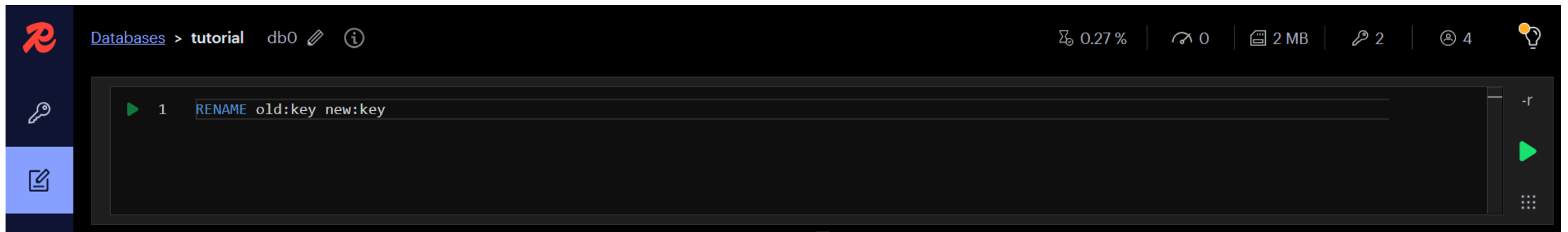
```
SET old:key value
```



Clear Results				
SET old:key value	Jun 19, 16:12:09	2.964 msec	📄	🗑️
"OK"				
SET auth:code:1001 482913 EX 180	Jun 19, 16:08:57	2.064 msec	🗑️	⏮️
SET shop:product:1001 "새 이름" XX	Jun 19, 16:03:49	0.8 msec	🗑️	⏮️

- 키 이름 변경

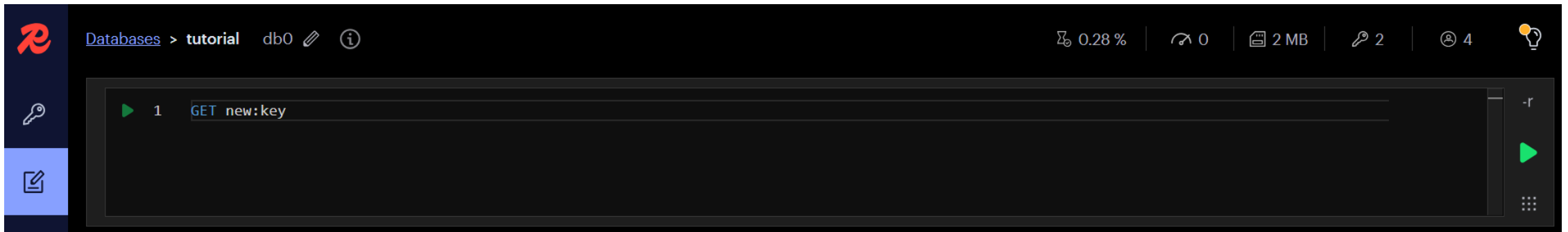
```
RENAME old:key new:key
```



Clear Results			
RENAME old:key new:key	Jun 19, 16:12:59	1.787 msec	   
"OK"			
SET old:key value	Jun 19, 16:12:09	2.964 msec	  
SET auth:code:1001 482913 EX 180	Jun 19, 16:08:57	2.064 msec	  

- 변경된 키 조회

GET new:key



Clear Results				
GET new:key	Jun 19, 16:13:53	0.942 msec	</> ▾	🗑️ ▶ ⌵
"value"				
RENAME old:key new:key	Jun 19, 16:12:59	1.787 msec		🗑️ ▶ ⌵
SET old:key value	Jun 19, 16:12:09	2.964 msec		🗑️ ▶ ⌵

5. 키 탐색: KEYS와 SCAN

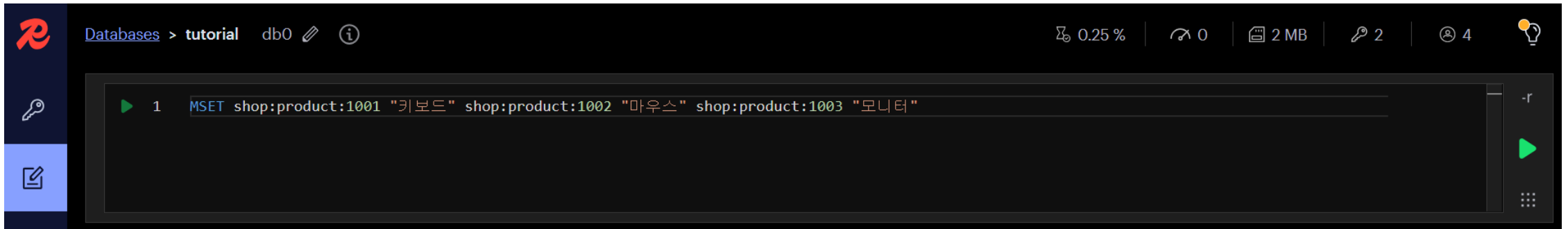
항목	KEYS	SCAN
목적	조건에 맞는 모든 Key 조회	Key를 조금씩 나누어 조회
반환 방식	한 번에 전체 반환	여러 번 호출하여 순차 반환
성능	$O(N)$ (전체 Key 탐색)	$O(1)$ (호출당 평균)
서버 영향	데이터가 많으면 Redis가 잠시 응답하지 않을 수 있음	서버 부하가 매우 적음

항목	KEYS	SCAN
운영 환경 사용	권장하지 않음	권장
개발/테스트	사용 가능	사용 가능
결과 개수	모든 Key	일부 Key (Cursor를 이용해 반복 조회)
Cursor 사용	없음	있음 (0 이 될 때까지 반복)
Pattern 검색	KEYS user:*	SCAN 0 MATCH user:*
COUNT 옵션	없음	있음 (COUNT 100 등)
대표 사용 사례	개발 중 데이터 확인	운영 서버에서 Key 조회

테스트용 데이터 생성

- **MSET** : 여러 개의 Key-Value를 한 번에 저장하는 명령어(Multiple SET)

```
MSET shop:product:1001 "키보드" shop:product:1002 "마우스" shop:product:1003 "모니터"
```

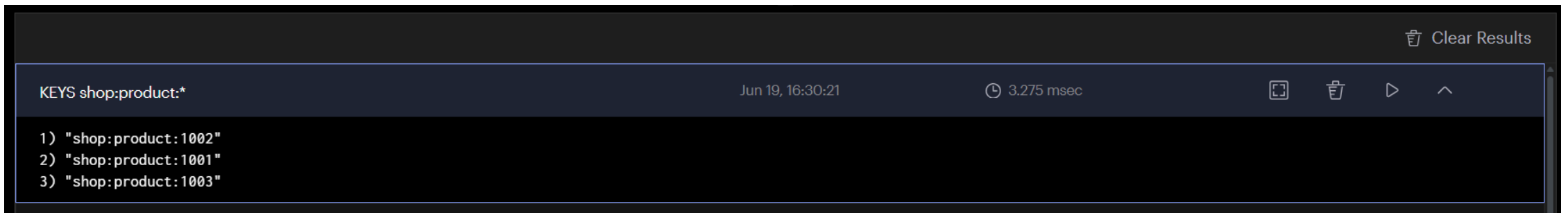
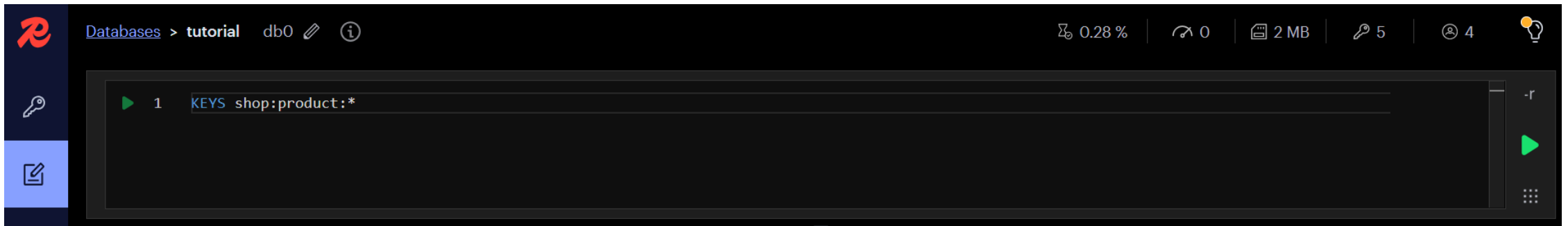


Clear Results			
MSET shop:product:1001 "키보드" shop:product:1002 "마우스" shop:product:1003 "모니터"	Jun 19, 16:28:33	3.9 msec	   
"OK"			
GET new:key	Jun 19, 16:13:53	0.942 msec	  
RENAME old:key new:key	Jun 19, 16:12:59	1.787 msec	  

KEYS

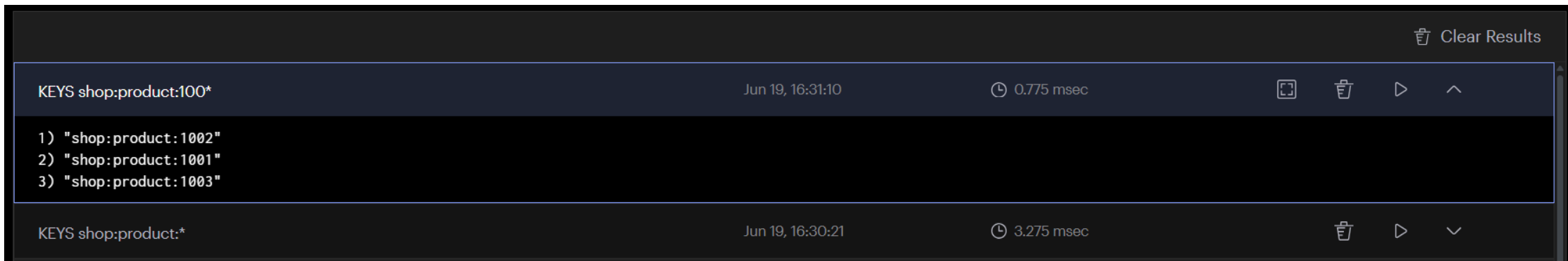
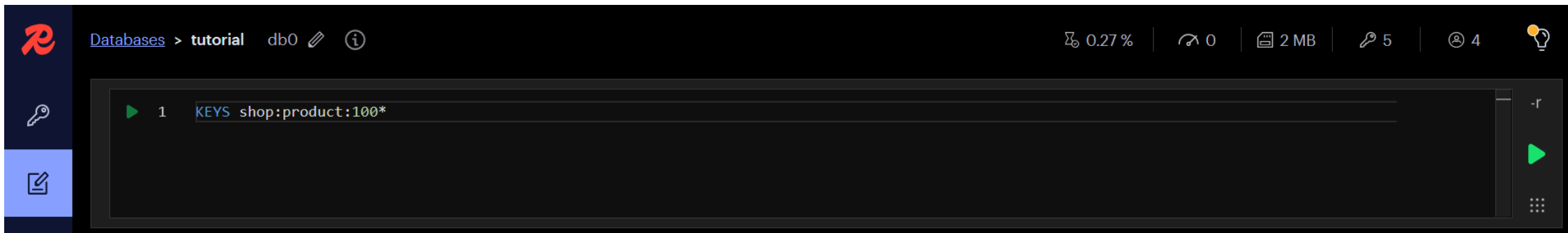
- 모든 상품 조회

```
KEYS shop:product:*
```



- 특정 상품 조회

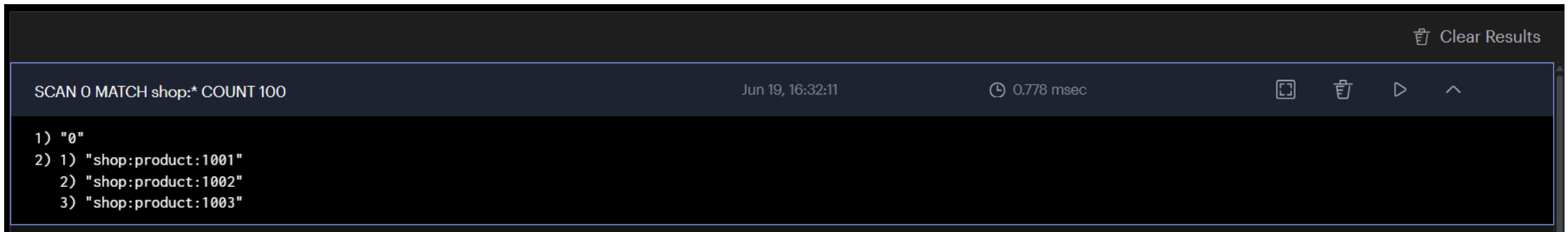
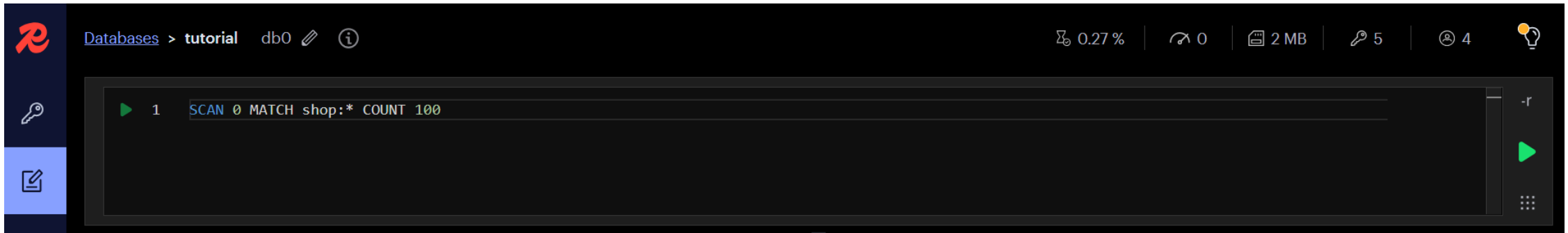
KEYS shop:product:100*



SCAN

- 모든 상품 조회

```
SCAN 0 MATCH shop:* COUNT 100
```



- 특정 상품 조회

```
SCAN 0 MATCH shop:product:1001 COUNT 10
```

